P, PI and PID Controllers

Embedded Control of Robotics – study notes (Lectures 6 and 7)

What a controller does

A controller drives a system towards a desired value called the setpoint. The difference between the setpoint and the measured output is the error, $e(t) = r(t) - y(t)$.

Example: an air conditioner set to 24 °C in a room at 30 °C has an error of 6 °C. The controller keeps cooling until the error is removed.

1. Proportional (P) controller

$$u(t) = K_p \cdot e(t)$$

- Acts on the present error only; Kp is the proportional gain.
- With Kp = 2 and e = 6, the output is 12, a strong cooling action.
- A larger error gives a larger response.
- Usually leaves a steady-state error: the output settles close to, but not exactly at, the setpoint.

2. Proportional–integral (PI) controller

$$u(t) = K_p \cdot e(t) + K_i \int e(t)\, dt$$

- P reacts to the present error; I adds up past error.
- The integral keeps growing while any error remains, so the steady-state error is removed.
- Response is slower and can show some overshoot.

3. Proportional–integral–derivative (PID) controller

$$u(t) = K_p \cdot e(t) + K_i \int e(t)\, dt + K_d \cdot \frac{de(t)}{dt}$$
$$C(s) = K_p + \frac{K_i}{s} + K_d \cdot s$$

- P – present error, I – accumulated past error, D – rate of change of the error (its trend).
- D damps the response and reduces overshoot.
- Fast, accurate and smooth, but there are three gains to tune.

Comparison

Controller	Uses	Steady-state error	Behaviour
P	Present error	Usually remains	Simple, fast
PI	Present + past	Removed	Slower, may overshoot

PD	Present + trend	Remains	Less overshoot, faster settling
PID	Present + past + trend	Removed	Fast, accurate, smooth

Analogy: holding a car at 60 km/h

- P: press the accelerator according to how far below 60 you are.
- PI: also press harder the longer you have stayed below 60.
- PID: also watch how fast the speed is changing, and ease off early so you do not overshoot.